

A Big Data Framework for Price Tracking and Product Matching in Sri Lankan E-Commerce

P.K.K. Dias

May 23, 2026

Chapter 1

Literature Review

Web crawling is the process of systematically browsing the web and extracting content from its pages. It is a fundamental technique used in various applications, most notably in search engines, where it is used to index the web and make it searchable. Another common use of web crawling is in data mining and web scraping, where it is used to collect data from websites for analysis or other purposes. More recently, web crawling has also been used in the context of training large language models, where it is used to gather vast amounts of text data from the web to train these models. Web crawlers, also known as spiders or bots, are automated programs that perform this task.

Crawlers typically traverse the web by following hyperlinks from one page to another, starting from a set of seed URLs [1]. They can be designed to crawl the entire web or to focus on specific domains, topics, or types of content.

A standard crawler operates in several stages [2]:

1. **Seed URLs:** The crawler starts with a list of initial URLs, known as seed URLs, which are the starting points for the crawling process.
2. **Downloading:** The crawler sends HTTP requests to the URLs and downloads the content of the web pages. The downloaded content is stored in local storage or cloud storage for further processing.
3. **Extracting links:** The crawler parses the downloaded pages to extract hyperlinks, which are then added to a crawl frontier.

A crawl frontier is a data structure that manages the list of URLs to be crawled, ensuring that the crawler does not revisit the same URL multiple times and that it adheres to any specified crawling policies (e.g., depth limits, domain restrictions). The crawl frontier can also be used as a queue to manage iterative and repeated crawling, allowing the crawler to revisit pages at regular intervals to check for updates, maintaining the freshness of the dataset [1].

Traditional web crawlers worked by sending HTTP requests to web servers and downloading the HTML content of web pages. However, such crawlers may struggle with modern websites that heavily rely on JavaScript and AJAX to render content dynamically via client-side rendering [3]. Modern web crawlers often need to use headless browsers or tools like Selenium, Puppeteer, or Playwright to handle JavaScript content and ensure that they can access the full content of web pages [4]. These tools allow the crawler to execute JavaScript and render the page as a regular browser would, while consuming fewer resources than a full browser, enabling the crawler to access dynamic content effectively.

It is also important to consider the legal and ethical implications of web crawling.

1. **Webmaster intent:** Webmasters may not want their content to be crawled or indexed, and they can use the `robots.txt` file to specify which parts of their site should not be accessed by crawlers [5].
2. **Server load:** Crawling can put a significant load on web servers, especially if the crawler is not designed to be respectful of the site's resources. It is important for crawlers to implement politeness policies, such as rate limiting, to avoid overwhelming servers [1].
3. **Data privacy:** Crawling and scraping can raise privacy concerns, especially if personal data is collected without consent. It is crucial to ensure that any data collected through crawling is handled in compliance with relevant data protection laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union [6].

A significant challenge in modern web crawling is the use of anti-bot measures by websites, such as CAPTCHAs, IP blocking, and rate limiting, which can hinder the crawling process [7]. Advanced security services such as Cloudflare employ fingerprinting techniques to identify and block automated traffic, making it increasingly difficult for crawlers to access content on certain

websites [8]. Crawlers need to be designed to navigate these challenges while still adhering to ethical and legal guidelines. Moreover, content hidden behind paywalls or login screens can also pose challenges for crawlers, as they may require authentication or subscription to access the content. Most crawlers, including Google’s [9] generally do not access such content, unless explicit permission and credentials are provided by the site owner.

1.0.1 Summary

Web crawling involves a range of technical and ethical considerations. Static crawlers are simple and efficient but fail on JavaScript-heavy pages, while headless browser-based crawlers handle dynamic content and client-side rendering at highest resource costs. Anti-bot measures and legal/ethical constraints add further complexity that must be carefully managed in any crawling system. Table 1.1 summarises the key crawler types and their trade-offs.

Crawler Type	Strengths	Limitations
Static HTTP crawler	Lightweight, fast, low resource usage	Cannot handle JavaScript / client-side rendering
Headless browser (Selenium, Puppeteer, Playwright)	Handles dynamic content; renders JS fully	High resource usage; slower at scale

Table 1.1: Summary of web crawler types and their trade-offs

Bibliography

- [1] M. Kumar, R. Bhatia, and D. Rattan, “A survey of web crawlers for information retrieval,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1218, 2017.
- [2] M. A. Kausar, V. Dhaka, and S. K. Singh, “Web crawler: a review,” *International Journal of Computer Applications*, vol. 63, no. 2, pp. 31–36, 2013.
- [3] A. Goel, J. Zhu, R. Netravali, and H. V. Madhyastha, “Sprinter: Speeding up High-Fidelity crawling of the modern web,” in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, (Santa Clara, CA), pp. 893–906, USENIX Association, Apr. 2024.
- [4] A. Mesbah, A. Van Deursen, and S. Lenseslink, “Crawling ajax-based web applications through dynamic analysis of user interface state changes,” *ACM Transactions on the Web (TWEB)*, vol. 6, no. 1, pp. 1–30, 2012.
- [5] Z. Chang, “A survey of modern crawler methods,” in *Proceedings of the 6th international conference on control engineering and artificial intelligence*, pp. 21–28, 2022.
- [6] M. A. Khder, “Web scraping or web crawling: State of art, techniques, approaches and application.,” *International Journal of Advances in Soft Computing & Its Applications*, vol. 13, no. 3, 2021.
- [7] K. Kothari, “Web crawler development: How to build a custom crawler,” Mar 2026.
- [8] Cloudflare, “Cloudflare bot management,” 2026.
- [9] Google, “Things to know about google’s web crawling — google crawling infrastructure — crawling infrastructure — google for developers,” 2026.